



Lecture 05

Classes and Objects

Basics of Internet Technologies 2

From loose data to meaningful models

Learning Objectives

- Explain the difference between a class and an object.
- Create classes with fields, properties, constructors, and methods.
- Use access modifiers: public and private.
- Explain instance members and static members.
- Use object references and lists of objects.
- Override ToString for readable console output.
- Save and load a List<Expense> as JSON.
- Replace dictionary-based expenses with an Expense class.

Connection to Lecture 04

- Lecture 04 stored one expense as Dictionary<string, string>.
- Dictionary keys such as name and amount are text and can be mistyped.
- The compiler cannot tell us that an expense should have an amount.
- Today we give the data a name, structure, and behavior: Expense.

Three-Hour Lesson Plan

- 20 min - problem with anonymous data; class and object vocabulary.
- 30 min - fields, properties, object creation, and constructors.
- 25 min - methods, ToString, and object references.
- 20 min - access modifiers and encapsulation.
- 20 min - static members and constructor overloads.
- 35 min - convert the expense tracker from dictionaries to Expense objects.
- 20 min - exercises, shared database increment, and homework briefing.

The Problem With a Dictionary

```
Dictionary<string, string> expense = new Dictionary<string, string>();  
expense["name"] = "Coffee";  
expense["amount"] = "4.50";  
expense["category"] = "Food";
```

- Every key is just a string.
- `expense["ammount"]` compiles but creates a different key.
- All values are strings, even the numeric amount.
- The structure is invisible in the type name.

Class and Object

Class

- A blueprint or template
- Defines data and behavior
- Example: Expense
- Written once in code

Object

- One value created from a class
- Has its own data
- Example: coffee expense
- Many objects can use one class

A Simple Expense Class

```
public class Expense
{
    public string Name { get; set; }
    public decimal Amount { get; set; }
    public string Category { get; set; }
}
```

- class defines a new C# type.
- Expense is now as real a type as string, int, or DateTime.
- The braces contain members belonging to that type.

Creating an Object

```
Expense coffee = new Expense();  
coffee.Name = "Coffee";  
coffee.Amount = 4.50m;  
coffee.Category = "Food";
```

- new creates an object in memory.
- coffee is a variable that refers to that object.
- The m suffix makes 4.50 a decimal literal.
- Each object can have different property values.

One Class, Many Objects

```
Expense coffee = new Expense();  
coffee.Name = "Coffee";
```

```
Expense bus = new Expense();  
bus.Name = "Bus ticket";
```

- coffee and bus have the same type.
- They are separate objects with separate data.
- A class is reusable; an object is one concrete instance.

Members Inside a Class

Data

- Fields
- Properties
- Read-only values

Behavior

- Methods
- Constructors
- Overrides such as ToString

Fields: Internal Variables

```
public class BankAccount
{
    private decimal balance;
}
```

- A field is a variable stored inside each object.
- `private` means code outside `BankAccount` cannot access `balance` directly.
- Fields are useful for internal implementation details.
- The object keeps its own state between method calls.

Properties: Controlled Data Access

```
public string Name { get; set; }  
public int Id { get; private set; }  
public string FullName => FirstName + " " + LastName;
```

- get reads a property value; set changes it.
- private set allows reading from outside but changes only inside the class.
- A get-only calculated property can derive a value from other properties.
- Properties are usually preferred for public data.

Auto-Property Sugar

```
public string Category { get; set; }  
  
// C# creates hidden storage for us.  
// No manual field is needed here.
```

- This is an auto-property.
- It is shorter than writing a private field plus get and set methods.
- Use it when no extra validation or logic is needed yet.

Property With Validation

```
private decimal amount;

public decimal Amount
{
    get { return amount; }
    set
    {
        if (value > 0) amount = value;
    }
}
```

- value is the incoming value in a property setter.
- The property can reject invalid state.
- Later we will discuss richer validation strategies.

Constructor: Start an Object

```
public Expense(int id, string name, decimal amount, string category)
{
    Id = id;
    Name = name;
    Amount = amount;
    Category = category;
}
```

- A constructor has the same name as its class.
- It runs automatically when new Expense(...) is used.
- Parameters provide the first values for the object.
- Constructors help avoid half-created objects.

Using the Constructor

```
Expense coffee = new Expense(  
    1,  
    "Coffee",  
    4.50m,  
    "Food");
```

- Arguments are matched to constructor parameters by position.
- The object is ready immediately after creation.
- Use meaningful parameter names to make construction easier to read.

Default Constructor

```
public Expense()  
{  
}  
  
public Expense(int id, string name, decimal amount, string category)  
{  
    // Set initial values.  
}
```

- A parameterless constructor has no parameters.
- System.Text.Json can use public properties when loading JSON.
- For this beginner project we keep both constructors.
- Later, design choices around construction become more deliberate.

Methods Belong to Objects

```
public bool IsLargeExpense()  
{  
    return Amount >= 100m;  
}  
  
if (coffee.IsLargeExpense())  
{  
    Console.WriteLine("Review this expense.");  
}
```

- An instance method works with one object's data.
- Inside the method, Amount means this object's Amount.
- Methods give behavior a meaningful name.

void Method vs Returning a Value

void

- Does an action
- Returns no result
- Example: `Enroll(course)`

Return type

- Calculates an answer
- Returns a value
- Example: `bool IsLargeExpense()`

ToString for Console Output

```
public override string ToString()
{
    return Id + ". " + Name + " - "
        + Amount.ToString("F2") + " EUR - " + Category;
}

Console.WriteLine(coffee);
```

- Every object has a ToString method inherited from object.
- override replaces the default type-name output.
- Console.WriteLine(coffee) calls coffee.ToString().

Object Reference: Important Mental Model

```
Student firstStudent = new Student("Ada", "Lovelace", 21);  
Student alias = firstStudent;  
  
alias.Age = 22;  
Console.WriteLine(firstStudent.Age); // 22
```

- Class objects are reference values.
- The two variables point to the same Student object.
- Changing through alias changes the shared object.
- This differs from copying an int or decimal value.

Visual Model: Reference

- firstStudent -> Student object { Name: Ada, Age: 22 }
- alias -> same Student object
- There is one object and two references to it.
- A new Student(...) expression would create another separate object.

List of Objects

```
List<Expense> expenses = new List<Expense>();  
expenses.Add(coffee);  
expenses.Add(bus);  
  
foreach (Expense expense in expenses)  
{  
    Console.WriteLine(expense.Name);  
}
```

- List<Expense> tells the compiler what every item must be.
- Autocomplete now knows properties such as Name and Amount.
- This is safer and clearer than List<Dictionary<string, string>>.

LINQ Becomes Clearer

```
decimal total = expenses.Sum(expense => expense.Amount);  
  
List<Expense> food = expenses  
    .Where(expense => expense.Category == "Food")  
    .ToList();
```

- The lambda receives an Expense object.
- expense.Amount is a decimal, not a text dictionary lookup.
- The domain model makes LINQ easier to read.

Access Modifiers

public

- Accessible from other code
- The class contract
- Use for intended operations

private

- Accessible only inside the class
- Protects implementation
- Use for internal fields and helpers

Encapsulation

```
public class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount)
    {
        if (amount > 0) balance += amount;
    }
}
```

- Outside code cannot write `account.balance = -100m`.
- The `Deposit` method decides which changes are valid.
- This keeps rules close to the data they protect.

Why Not Make Everything Public?

- Public state can be changed from anywhere.
- Rules become scattered across Program.cs and other files.
- A later change can break many callers.
- Expose the smallest useful public surface.
- For simple teaching models, auto-properties are acceptable; learn the trade-off.

Static Members

```
public class Student
{
    public static int CreatedCount { get; private set; }

    public Student(...)
    {
        CreatedCount++;
    }
}

Console.WriteLine(Student.CreatedCount);
```

- static belongs to the class itself, not to one object.
- Student.CreatedCount is shared by all Student objects.
- Use the class name to access a static member.

Instance vs Static

Instance

- Belongs to one object
- `coffee.Amount`
- Each object has its own value

Static

- Belongs to the class
- `Student.CreatedCount`
- One shared value

Constructor Overloading

```
public Book(string title)
    : this(title, "Unknown", 0)
{
}

public Book(string title, string author, int year)
{
    Title = title;
    Author = author;
    Year = year;
}
```

- Overloading means using the same constructor name with different parameter lists.
- `this(...)` calls another constructor in the same class.
- It prevents repeating initialization code.

readonly Field

```
private readonly List<string> courses =  
    new List<string>();
```

- readonly means the field reference is assigned only during declaration or construction.
- The list object can still receive items with `courses.Add(...)`.
- It prevents replacing the list by mistake later.

Calculated Property

```
public string FullName
{
    get { return FirstName + " " + LastName; }
}

// Short form:
public string FullName => FirstName + " " + LastName;
```

- A calculated property does not store another copy of the full name.
- It computes the current result when read.
- The short expression-bodied syntax is C# sugar for a simple getter.

Class Design Checklist

- Choose a noun for the class: Expense, Student, Book, User.
- Give the object only data that belongs to that concept.
- Use meaningful types: decimal for money, DateTime for dates.
- Add a constructor when an object needs required starting data.
- Put behavior near the data it works with.
- Keep the public API small and understandable.

From Dictionary to Expense

Before

- Dictionary<string, string>
- expense["amount"]
- Every value is text
- Keys can be mistyped

After

- Expense
- expense.Amount
- Amount is decimal
- Compiler knows the model

Expense Model Used in the Demo

```
public class Expense
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Amount { get; set; }
    public string Category { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

- Id identifies an expense in the console list.
- Name and Category are descriptive text.
- Amount uses decimal because this is money.
- CreatedAt records when the item was added.

JSON Still Works With Classes

```
List<Expense> loaded =  
    JsonSerializer.Deserialize<List<Expense>>(json);  
  
string json = JsonSerializer.Serialize(expenses,  
    new JsonSerializerOptions { WriteIndented = true });
```

- JSON property names are matched with public properties.
- The file becomes easier to understand because Expense is explicit.
- The persistence idea from Lecture 04 stays the same.

Live Coding Sequence

- 1. Create the Expense class with five properties.
- 2. Change `List<Dictionary<string, string>>` to `List<Expense>`.
- 3. Change `AddExpense` to call `new Expense(...)`.
- 4. Replace `expense["name"]` with `expense.Name`.
- 5. Add `ToString` and simplify console output.
- 6. Run, save JSON, restart, and verify loading.

Common Beginner Errors

- Using `Expense.Name` instead of `coffee.Name` for an instance property.
- Forgetting `new` before `Expense(...)`.
- Writing a constructor with a return type such as `void`.
- Expecting a class variable copy to create a new object.
- Making a field `public` only to avoid writing a method or property.
- Using `double` for money instead of `decimal`.

Guided Exercise A

- Create a Book class: Title, Author, Year.
- Add a constructor that receives all three values.
- Create two Book objects.
- Override ToString to show a readable sentence.
- Put both books into List<Book> and print them with foreach.

Guided Exercise B

- Create a Student class with FirstName, LastName, and Age.
- Add a FullName calculated property.
- Add a method Introduce that returns a string.
- Create a static CreatedCount member.
- Explain which data belongs to one object and which belongs to the class.

Shared Database-Course Increment

- Use the same project domain as the backend application.
- Design an Expense table with a primary key and five columns.
- Choose SQL types for id, name, amount, category, and created date.
- Write a short note explaining why Amount is numeric, not text.
- The backend still saves JSON today; this schema prepares the database milestone.

Homework: Project Evolution Portfolio

- Upgrade Milestone 01 from dictionary-based data to classes.
- Keep the menu operations: add, list, search, filter, delete, and summary.
- Persist List<YourEntity> in JSON.
- Submit the full solution and a README explaining what changed in Lecture 05.
- This is one step in Assignment 1, worth 30 percent of the course grade.

Exit Ticket

- What is the difference between a class and an object?
- Why is decimal appropriate for an expense amount?
- When would private be better than public?
- What does new Expense(...) do?
- What will ToString change in the console application?



Next Time

Interfaces and Abstraction

We will compare different implementations behind one common contract.
The Expense class becomes part of a larger design.